

SEIP Assignment 1: Final Submission Report

Advanced DevOps: Production-Grade CI/CD, External Configuration, and Orchestration

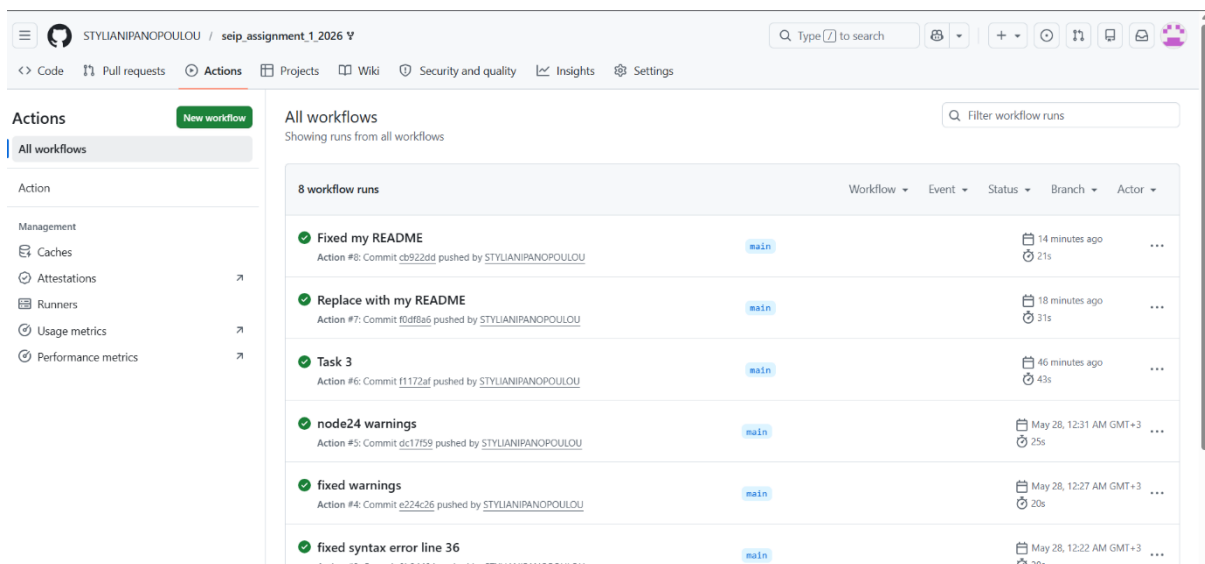
1. GitHub Repository

The public repository containing all source code, the GitHub Actions workflow file, and the complete k8s/ directory can be found at the link below.

[STYLIANIPANOPOULOU/seip_assignment_1_2026](https://github.com/STYLIANIPANOPOULOU/seip_assignment_1_2026)

2. CI/CD Pipeline : Proof of Success

The screenshot below shows the GitHub Actions dashboard after a successful pipeline run triggered by a push to the main branch. The workflow built the Docker image, authenticated to GHCR using the built-in GITHUB_TOKEN, and pushed the final image



3. Cluster State : Proof of Healthy Deployment

3.1 kubectl get all -n default

The output below shows 3 running pods, the deployment, the replica set, and the ClusterIP service all in a healthy state.

```
C:\cygwin64\home\stell\seip_assignment_1_2026>kubectl get all -n default
NAME                                     READY   STATUS    RESTARTS   AGE
pod/seip-assignment-1-deployment-6b45b946c6-bwf7x  1/1     Running   1 (2m1s ago)  4d23h
pod/seip-assignment-1-deployment-6b45b946c6-kfc2n  1/1     Running   1 (2m1s ago)  4d23h
pod/seip-assignment-1-deployment-6b45b946c6-zwdpr  1/1     Running   1 (2m1s ago)  4d23h

NAME                                     TYPE          CLUSTER-IP   EXTERNAL-IP   PORT(S)    AGE
service/kubernetes                      ClusterIP     10.96.0.1    <none>        443/TCP    6d19h
service/seip-assignment-1-service       ClusterIP     10.98.179.94 <none>        80/TCP     4d23h

NAME                                     READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/seip-assignment-1-deployment  3/3     3             3           4d23h

NAME                                     DESIRED   CURRENT   READY   AGE
replicaset.apps/seip-assignment-1-deployment-6b45b946c6  3         3         3       4d23h
```

3.2 kubectl get configmap,secret

The output below confirms that both the ConfigMap and the Secret were applied successfully to the cluster.

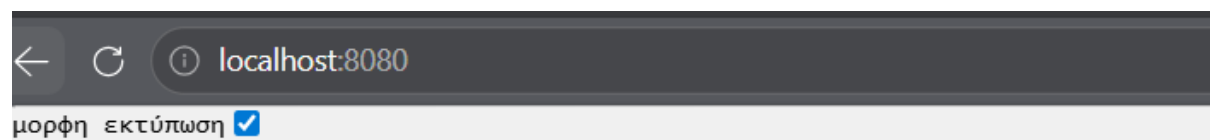
```
C:\cygwin64\home\stell\seip_assignment_1_2026>kubectl get configmap,secret
NAME                                DATA  AGE
configmap/kube-root-ca.crt          1      6d19h
configmap/seip-assignment-1-config  2      4d23h

NAME                                TYPE    DATA  AGE
secret/seip-assignment-1-secret     Opaque  1      4d23h
```

4. Application Verification

4.1 Root endpoint — http://localhost:8080/

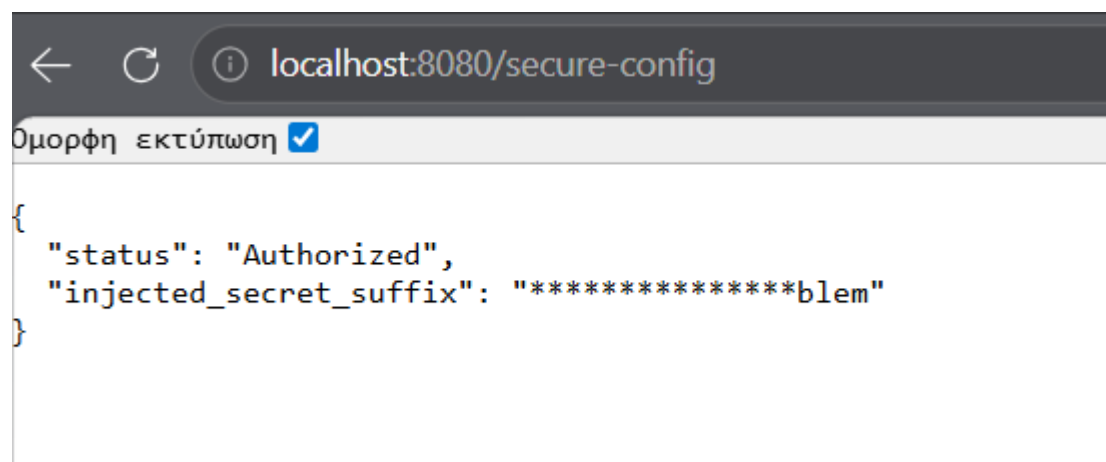
The response below shows the custom greeting message that was injected into the container from the ConfigMap at runtime.



```
"message": "Welcome to the Software Engineering in Practice Assignment!",
"environment": "production"
```

4.2 Secure config endpoint — http://localhost:8080/secure-config

The response below confirms the status is "Authorized" and shows the masked suffix of the API secret that was injected from the Kubernetes Secret.



5. AI Usage & Future Engineering Report

5.1 AI Integration

For this assignment, I used Google Gemini as a kind of digital tutor rather than a tool that writes code for me. Since I am still learning the basics of Cloud-Native technologies and DevOps, my main goal was to actually understand what I was doing, not just copy-paste commands. I used the AI to explain the differences between various Kubernetes components in plain terms — for example, what a ClusterIP actually does in practice and to walk me through the command-line steps one at a time so I didn't feel completely lost.

5.2 Utility Analysis

The most helpful thing was using the AI as an interactive documentation tool. Instead of reading through the full Kubernetes docs to understand one field, I could ask a targeted question and get a focused answer. It was particularly useful for understanding how Base64 encoding works for Secrets, why the value needs to be encoded in the first place and how Kubernetes decodes it back when injecting it into the container. Having that explained in simple terms helped me do it correctly myself instead of just copying a command I didn't understand.

It also helped me understand the connection between the YAML files and the actual application. Seeing how a key in configmap.yaml maps directly to an environment variable that server.js reads at boot made me understand better everything .

5.3 Friction Points

There were a couple of moments when the AI's guidance left me stuck because it assumed my local environment was already perfectly configured. The biggest one was when I applied the manifests and got a wall of connection errors saying, "failed to connect to the docker API". The YAML code itself was fine, the problem was that the AI never told me I needed to have Docker Desktop running first. I only figured it out by reading the error messages.

Another confusing moment was with kubectl port-forward. After I ran the command, the terminal just froze and I thought something had crashed. It took me a minute to realize the command is supposed to block the terminal.

AI also occasionally suggested slightly outdated API field names for Kubernetes resources. Nothing that broke anything, but I learned to double-check against the official docs when something didn't apply correctly.

5.4 Future Architectural Outlook

If I had more time and this were heading toward a real production setup, there are a few things I would want to add. The first would be replacing the manual port-forward with a proper Ingress Controller. Right now, keeping a terminal open just to access the app doesn't scale at all, an Ingress would handle routing properly without that workaround.

I would also look into setting up GitOps workflow with ArgoCD. The idea of the cluster automatically syncing itself to whatever is in the GitHub repository, instead of me running kubectl applied manually every time, sounds like the right way to do continuous delivery properly.

Finally, adding Prometheus for monitoring would make a big difference. Right now, I have no visibility into what's actually happening inside the pods, how much memory they're using, whether requests are failing, and so on. Having that data in real time would be essential for any real deployment.